

Migration Testing & Validation

Structured Test Plans for Zero-Defect Cutover

A failed migration test costs hours. A failed production cutover costs thousands. This guide provides structured test plans at every stage — pre-migration readiness, conversion verification, boot testing, application smoke tests, performance baselines, and acceptance sign-off. Every test is automatable with `h2kvmctl`.

Test Plans | Boot Test | Smoke Test | Performance Baseline | Acceptance Criteria | Automated Validation

Testing Framework

Five testing stages ensure zero-defect migration from VMware to KVM.

Migration Testing Stages

Stage	Test Type	When	Duration	What It Validates	Tool	Pass Criteria
1	Pre-flight Readiness	Before migration	5 min	Host capabilities, disk space, packages, network	h2kvmctl doctor	All checks pass (0 failures)
2	Dry-Run Validation	Before migration	2 min/VM	YAML config, source files, dependencies, space	h2kvmctl --dry-run	Zero errors, all sources found
3	Conversion Verification	After conversion	1 min/VM	Output file integrity, size, format, checksum	qemu-img check	No corruption, checksum match
4	Boot Test	After deploy	2 min/VM	VM boots, gets IP, SSH accessible, OS functional	h2kvmctl boot-test	VM reaches login prompt in <120s
5	Application Smoke Test	After boot	5-30 min/VM	Application responds, data intact, services running	Custom scripts	All endpoints respond correctly
6	Performance Baseline	After smoke test	15 min/VM	CPU, memory, disk I/O, network match or exceed VMware	sysbench, fio, iperf3	Within 10% of VMware baseline
7	Acceptance Sign-off	After all tests	1 day	App owner verifies functionality in production	Manual + monitoring	App owner signs acceptance form

Test Automation Coverage

Stages 1-4 are fully automated by h2kvmctl. Stage 5 requires custom scripts per application (templates provided). Stage 6 uses standard benchmarking tools. Stage 7 is a manual sign-off. **Target: 85%+ test automation coverage.**

Pre-Flight & Dry-Run Tests

Catch problems before they happen — validate everything before touching production.

Pre-Flight Checklist (Automated)

Host Readiness (h2kvmctl doctor)

```
# Run on each target KVM host
h2kvmctl doctor --json | jq '.checks[] | select(.status != "pass")'

# Critical checks:
# - KVM module loaded
# - QEMU version >= 6.0
# - libvirtd running
# - Disk space >= 2x largest VM
# - Network bridge exists + UP
# - VirtIO drivers available
# - OVMF firmware installed

# Gate: ALL critical checks must pass
# Warning: non-critical can proceed with note
```

Dry-Run Validation

```
# Validate migration config without executing
h2kvmctl convert --config batch.yaml --dry-run --json

# What dry-run checks:
# 1. YAML syntax valid
# 2. All source files exist and readable
# 3. Source format recognized (VMDK/OVA/VHD)
# 4. Output directory writable
# 5. Sufficient disk space for all VMs
# 6. Network bridges referenced exist
# 7. Dependency ordering valid (no cycles)
# 8. Estimated conversion time

# Example output:
# { "status": "ready",
#   "vms": 15,
#   "total_source_size_gb": 750,
#   "estimated_output_gb": 450,
#   "available_space_gb": 1200,
#   "estimated_duration_minutes": 95 }
```

Source Image Validation

Check	Command	Pass Criteria	Failure Action
File integrity	qemu-img check source.vmdk	No errors, 0 leaked clusters	Re-export from VMware; if persistent, repair with --repair=all
Format detection	qemu-img info source.vmdk	Format correctly identified	Specify format explicitly: -f vmdk
Size reasonable	qemu-img info --output=json	Virtual size matches expected	Verify correct VM was exported
OVA structure	tar tf export.ova	Contains .ovf + .vmdk files	OVA corrupted during download; re-export

Checksum	sha256sum source.vmdk	Matches source manifest	File corrupted in transfer; re-copy
----------	-----------------------	-------------------------	-------------------------------------

Boot Test Automation

Automatically verify every VM boots successfully after conversion.

Automated Boot Test Script

```
#!/bin/bash
# boot-test.sh – Verify VM boots and reaches network
VM_NAME="$1"
TIMEOUT=120
BRIDGE="br0"

echo "[TEST] Starting boot test for $VM_NAME..."

# Start VM
virsh start "$VM_NAME" 2>/dev/null
if [ $? -ne 0 ]; then echo "[FAIL] Cannot start $VM_NAME"; exit 1; fi

# Wait for VM to get IP address
echo "[TEST] Waiting for IP address (timeout: ${TIMEOUT}s)..."
for i in $(seq 1 $TIMEOUT); do
    IP=$(virsh domifaddr "$VM_NAME" 2>/dev/null | grep -oP '(\d+\.){3}\d+')
    if [ -n "$IP" ]; then
        echo "[PASS] $VM_NAME got IP: $IP (${i}s)"
        break
    fi
    sleep 1
done

if [ -z "$IP" ]; then echo "[FAIL] No IP after ${TIMEOUT}s"; virsh destroy "$VM_NAME"; exit 1; fi

# Test SSH connectivity (Linux VMs)
echo "[TEST] Testing SSH connectivity..."
for i in $(seq 1 30); do
    ssh -o ConnectTimeout=3 -o StrictHostKeyChecking=no "$IP" "hostname" &>/dev/null
    if [ $? -eq 0 ]; then echo "[PASS] SSH accessible at $IP"; exit 0; fi
    sleep 2
done

# Test RDP connectivity (Windows VMs)
nc -z -w 3 "$IP" 3389 &>/dev/null
if [ $? -eq 0 ]; then echo "[PASS] RDP accessible at $IP (Windows)"; exit 0; fi

echo "[WARN] VM booted with IP but SSH/RDP not accessible"
exit 2
```

Boot Test Results Matrix

#	VM Name	OS	Boot Time	Got IP?	SSH/RDP?	Result	Notes
1	db-server	RHEL 9	18s	Yes	Yes	PASS	Clean boot, all services started
2	app-server-01	Ubuntu 22	22s	Yes	Yes	PASS	Cloud-init disabled successfully
3	web-frontend	Rocky 9	15s	Yes	Yes	PASS	nginx responding on port 80
4	win-server-01	Win 2022	45s	Yes	RDP	PASS	VirtIO drivers loaded, RDP accessible
5	legacy-app	CentOS 7	35s	Yes	No	WARN	SSH key auth changed; need password

Application Smoke Tests

Verify applications work correctly after migration — templates for common services.

Smoke Test Templates

Web Application

```
#!/bin/bash
# smoke-test-web.sh
VM_IP="$1"
echo "[TEST] Web app smoke test..."

# HTTP response
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" \
  "http://$VM_IP/")
[ "$HTTP_CODE" = "200" ] && echo "[PASS] HTTP 200" \
  || echo "[FAIL] HTTP $HTTP_CODE"

# HTTPS
HTTP_CODE=$(curl -sk -o /dev/null -w "%{http_code}" \
  "https://$VM_IP/")
[ "$HTTP_CODE" = "200" ] && echo "[PASS] HTTPS 200" \
  || echo "[FAIL] HTTPS $HTTP_CODE"

# API health endpoint
HEALTH=$(curl -s "http://$VM_IP/api/health" | jq -r '.status')
[ "$HEALTH" = "ok" ] && echo "[PASS] API healthy" \
  || echo "[FAIL] API: $HEALTH"
```

Database Server

```
#!/bin/bash
# smoke-test-db.sh
VM_IP="$1"
echo "[TEST] Database smoke test..."

# PostgreSQL connection
psql -h "$VM_IP" -U app_user -d appdb -c "SELECT 1" \
  &>/dev/null && echo "[PASS] PostgreSQL connected" \
  || echo "[FAIL] PostgreSQL connection failed"

# Row count check (compare to pre-migration)
COUNT=$(psql -h "$VM_IP" -U app_user -d appdb -tAc \
  "SELECT count(*) FROM orders")
PRE_COUNT=1542387 # from pre-migration baseline
[ "$COUNT" = "$PRE_COUNT" ] \
  && echo "[PASS] Row count matches ($COUNT)" \
  || echo "[FAIL] Row count mismatch: $COUNT vs $PRE_COUNT"

# Replication status (if replica)
psql -h "$VM_IP" -U postgres -c "SELECT * FROM pg_stat_replication" \
  &>/dev/null && echo "[PASS] Replication active"
```

Windows / Active Directory

```
# smoke-test-windows.ps1 (run via WinRM)
# RDP accessibility
Test-NetConnection -ComputerName $VM_IP -Port 3389
# [PASS] RDP port open

# Domain services (if DC)
Get-ADDomainController -Filter *
# [PASS] AD DS responding

# DNS resolution
```

Generic Service Check

```
#!/bin/bash
# smoke-test-services.sh
VM_IP="$1"
SERVICES_FILE="$2" # services.txt

# Test each service port
while IFS=, read -r name port protocol; do
  if [ "$protocol" = "tcp" ]; then
    nc -z -w 3 "$VM_IP" "$port" &>/dev/null
    [ $? -eq 0 ] \
```

```
Resolve-DnsName -Name corp.local -Server $VM_IP
# [PASS] DNS resolution working

# File share accessibility
Test-Path "\\$VM_IP\shared$"
# [PASS] File share accessible

# SQL Server (if applicable)
Invoke-Sqlcmd -ServerInstance $VM_IP \
-Query "SELECT @@VERSION"
# [PASS] SQL Server responding
```

```
&& echo "[PASS] $name (TCP/$port)" \
|| echo "[FAIL] $name (TCP/$port)"
fi
done < "$SERVICES_FILE"

# Example services.txt:
# ssh,22,tcp
# http,80,tcp
# https,443,tcp
# postgresql,5432,tcp
# redis,6379,tcp
# elasticsearch,9200,tcp
```


Performance Baseline

Measure before and after — prove KVM matches or exceeds VMware performance.

Benchmark Suite

Metric	Tool	Command	VMware Baseline	KVM Target	Pass Criteria
CPU (single)	sysbench	sysbench cpu --threads=1 run	1,250 events/s	1,250+ events/s	Within 5%
CPU (multi)	sysbench	sysbench cpu --threads=8 run	9,800 events/s	9,800+ events/s	Within 5%
Memory BW	sysbench	sysbench memory --threads=4 run	8.5 GB/s	8.5+ GB/s	Within 10%
Disk IOPS	fio	fio --name=test --rw=randread --bs=4k --iodepth=32 --runtime=30	45,000 IOPS	45,000+ IOPS	Within 10%
Disk throughput	fio	fio --name=test --rw=read --bs=1M --iodepth=8 --runtime=30	1.2 GB/s	1.2+ GB/s	Within 10%
Disk latency	fio	fio --name=test --rw=randread --bs=4k --iodepth=1 --runtime=30	0.15ms	0.15ms or less	Within 20%
Network BW	iperf3	iperf3 -c target -t 30	9.2 Gbps	9.2+ Gbps	Within 5%
Network latency	ping	ping -c 100 target	0.3ms avg	0.3ms avg	Within 20%

Performance Comparison Report

```
# Generate performance comparison report
h2kvmctl perf-report --baseline vmware-baseline.json --current kvm-results.json

# Output:
# Performance Comparison: db-server
# =====
# Metric          VMware      KVM         Delta      Status
# CPU single-thread 1,250 ev/s  1,285 ev/s  +2.8%     PASS
# CPU multi-thread  9,800 ev/s  10,100 ev/s +3.1%     PASS (better)
# Memory bandwidth  8.5 GB/s    9.1 GB/s    +7.1%     PASS (better)
# Disk IOPS (4K)    45,000      52,000      +15.6%    PASS (better)
# Disk throughput   1.2 GB/s    1.4 GB/s    +16.7%    PASS (better)
# Disk latency      0.15ms      0.12ms      -20.0%    PASS (better)
# Network bandwidth 9.2 Gbps    9.5 Gbps    +3.3%     PASS (better)
# Network latency   0.3ms       0.28ms      -6.7%     PASS (better)
```

#

Overall: 8/8 PASS – KVM outperforms VMware on all metrics

Acceptance Criteria & Sign-off

Formal acceptance process with documented sign-off from stakeholders.

Acceptance Criteria Template

#	Criterion	Priority	Verification Method	Status	Sign-off By
1	VM boots within 2 minutes	Critical	Automated boot test timer		Migration Lead
2	All services start automatically	Critical	Service port check script		Migration Lead
3	Application responds to requests	Critical	Smoke test script		App Owner
4	Data integrity verified (row counts, checksums)	Critical	Database validation script		DBA / App Owner
5	Network connectivity to all dependencies	Critical	Connectivity matrix test		Network Admin
6	Performance within 10% of VMware baseline	High	Benchmark comparison report		Migration Lead
7	Monitoring agents reporting (Prometheus, Zabbix)	High	Dashboard verification		Ops Team
8	Backup schedule configured and tested	High	Backup test restore		Backup Admin
9	DNS records updated (if IP changed)	High	nslookup verification		Network Admin
10	Rollback procedure documented and tested	Medium	Rollback drill in pre-prod		Migration Lead

Go/No-Go Decision Matrix

- **GO:** All Critical criteria pass + 80%+ High criteria pass
- **CONDITIONAL GO:** All Critical pass, some High fail with workaround
- **NO-GO:** Any Critical criterion fails
- Decision made 1 hour before cutover window
- Decision authority: Migration Lead + App Owner + Ops Manager
- No-Go = rollback to VMware, retry next window

Sign-off Template

MIGRATION ACCEPTANCE SIGN-OFF

VM Name: _____
Migration Date: _____
Change Ticket: _____

I confirm that:
☐ Application functionality verified

- ☐ Data integrity confirmed
- ☐ Performance acceptable
- ☐ Monitoring in place
- ☐ Backup configured

App Owner: _____

Signature: _____

Date: _____

Migration Lead: _____

Signature: _____

Date: _____

Regression & Soak Testing

Extended validation to catch issues that only appear under sustained load.

24-Hour Soak Test

- Run VM under production-like load for 24 hours
- Monitor: CPU, memory, disk I/O, network, swap usage
- Check for: memory leaks, disk space growth, log rotation
- Verify: no kernel panics, no OOM kills, no service crashes
- Compare metrics to VMware 24-hour baseline
- Tool: `stress-ng --cpu 4 --vm 2 --vm-bytes 2G --timeout 86400`
- Gate: zero errors in 24 hours = pass

Failover Testing

- Kill VM process unexpectedly: `kill -9 $(pgrep qemu)`
- Verify Pacemaker restarts VM on same or different host
- Measure RTO: time from failure to VM responding
- Test live migration: `virsh migrate --live vm host2`
- Verify application state preserved after migration
- Test storage failover (if Ceph): kill OSD, verify recovery
- Gate: RTO < 5 minutes for automated failover

Network Partition Test

- Simulate network outage to VM: `iptables -A FORWARD -d $VM_IP -j DROP`
- Verify application handles gracefully (timeouts, retries)
- Restore network: `iptables -D FORWARD -d $VM_IP -j DROP`
- Verify application recovers automatically
- Check database replication catches up
- Verify monitoring alerts fired during outage
- Gate: application recovers within 60 seconds of network restore

Backup / Restore Test

- Take backup of migrated VM
- Restore to a different name/location
- Boot restored VM, verify data integrity
- Measure restore time (RTO for backup recovery)
- Test with: QCOW2 snapshot, BorgBackup, Velero
- Verify backup encryption works correctly
- Gate: successful restore with data verification

Test Automation Pipeline

Integrate all tests into CI/CD for fully automated migration validation.

Automated Test Pipeline

```
# .gitlab-ci.yml - Migration Test Pipeline
stages: [preflight, convert, boot-test, smoke-test, perf-test, accept]

preflight:
  stage: preflight
  script:
    - h2kvmctl doctor --json > doctor-report.json
    - h2kvmctl convert --config $CONFIG --dry-run --json > dry-run.json
    - "[ $(jq -r '.status' dry-run.json) = 'ready' ]"
  artifacts: { paths: [doctor-report.json, dry-run.json] }

convert:
  stage: convert
  script:
    - h2kvmctl convert --config $CONFIG --json > conversion.json
    - "[ $(jq -r '.status' conversion.json) = 'success' ]"
  artifacts: { paths: [conversion.json] }

boot-test:
  stage: boot-test
  script:
    - for vm in $(jq -r '.vms[].name' $CONFIG); do ./boot-test.sh "$vm"; done
  allow_failure: false

smoke-test:
  stage: smoke-test
  script:
    - for vm in $(jq -r '.vms[].name' $CONFIG); do ./smoke-test.sh "$vm"; done

perf-test:
  stage: perf-test
  script:
    - ./perf-baseline.sh --compare vmware-baseline.json --output kvm-results.json
    - h2kvmctl perf-report --baseline vmware-baseline.json --current kvm-results.json

accept:
  stage: accept
  script:
    - echo "All automated tests passed. Manual acceptance required."
```

```
- h2kvmctl evidence-package --dir evidence/ --sign  
when: manual # Requires human approval
```

Automated Testing = Confident Cutover. Every Time.

With 7 test stages and 85% automation, you know before cutover that every VM will boot, every application will respond, and performance will match or exceed VMware. The evidence package satisfies auditors, the go/no-go matrix protects against risk, and the CI/CD pipeline makes it repeatable for every migration wave. Test once, trust always.